

onlinetail2_1

```
%pyspark
df_zeppelin = spark.read.parquet("/onlineretail2/onlineretail_ii_v1.parquet")
```

READY

```
%pyspark
df_zeppelin.show(5)
```

READY

[Invoice StockCode	Description	Quantity	InvoiceDate	Price	Customer_ID	Country	Total_Sales	Log_Total_Sales
[489434	85048 15CM CHRISTMAS GL...	12	2009-12-01 07:45:00	6.95	13085.0	United Kingdom	83.4	1.9263424466256551
[489434	79323P PINK CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.0	United Kingdom	81.0	1.9138138523837167
[489434	79323W WHITE CHERRY LIGHTS	12	2009-12-01 07:45:00	6.75	13085.0	United Kingdom	81.0	1.9138138523837167
[489434	22041 RECORD FRAME 7" S...	48	2009-12-01 07:45:00	2.1	13085.0	United Kingdom	100.80000000000001	2.007747778000474
[489434	21232 STRAWBERRY CERAMI...	24	2009-12-01 07:45:00	1.25	13085.0	United Kingdom	30.0	1.4913616938342726
only showing top 5 rows								

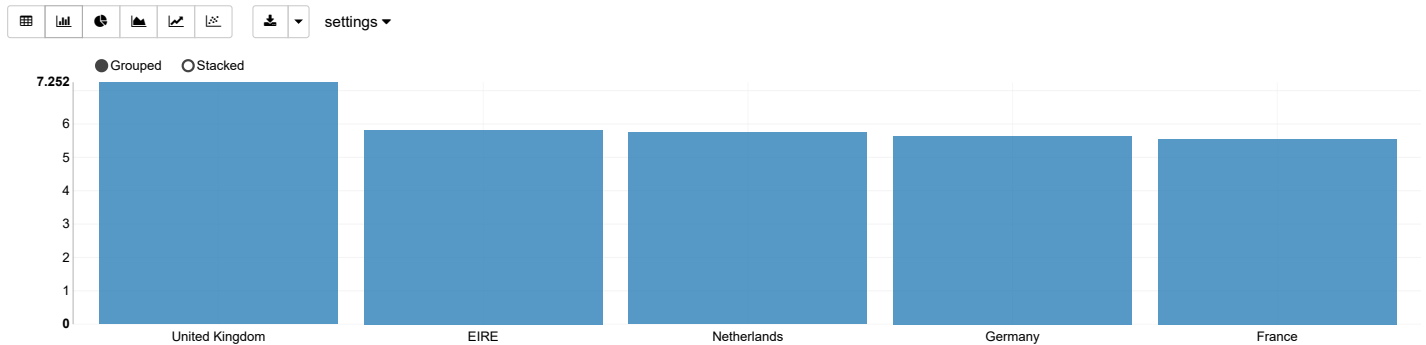
only showing top 5 rows

```
%pyspark
#convert to SQL Table view
df_zeppelin.createOrReplaceTempView("online_retail")
```

READY

```
%sql
--TESTING
SELECT Country, LOG10(SUM(Total_Sales)) as Log_Total_Sales
FROM online_retail
GROUP BY Country
ORDER BY Log_Total_Sales DESC
LIMIT 10
```

READY



```

%sql
--Check Missing Values
SELECT
COUNT(*) - COUNT(Invoice) AS Invoice_null,
COUNT(*) - COUNT(StockCode) AS StockCode_null,
COUNT(*) - COUNT(Description) AS Description_null,
COUNT(*) - COUNT(Quantity) AS Quantity_null,
COUNT(*) - COUNT(InvoiceDate) AS InvoiceDate_null,
COUNT(*) - COUNT(Price) AS Price_null,
COUNT(*) - COUNT(Customer_ID) AS CustomerID_null,
COUNT(*) - COUNT(Country) AS Country_null
FROM online_retail

```

READY



```
%sql
--create lookup table to map the description
CREATE OR REPLACE TEMPORARY VIEW stock_desc AS
SELECT
    StockCode,
    MAX(Description) AS Description
FROM online_retail
WHERE Description IS NOT NULL AND Description != ''
GROUP BY StockCode
```

READY



%sql

--impute the missing value
CREATE OR REPLACE TEMPORARY VIEW online_retail_cleaned AS
SELECT
 main.Invoice,
 main.StockCode,
 COALESCE(
 NULLIF(main.Description, ''),
 lookup.Description,
 'NA'
) AS Description,
 main.Quantity,
 main.InvoiceDate,
 main.Price,
 main.Customer_ID,
 main.Country
FROM online_retail main
LEFT JOIN stock_desc lookup
ON main.StockCode = lookup.StockCode

READY

%sql

--review
SELECT COUNT(*) AS remaining_missing_descriptions
FROM online_retail_cleaned
WHERE Description IS NULL OR Description = ''

settings

No Data Available

%sql

-- Check if description ada unique ke
SELECT
 StockCode,
 COUNT(DISTINCT Description) AS Total_Unique_Descriptions,
 COLLECT_SET(Description) AS List_Of_Descriptions
FROM online_retail
GROUP BY StockCode
HAVING COUNT(DISTINCT Description) > 1

StockCode

Total_Unique_Descriptions

%sql

READY

-- clean dup product name

CREATE OR REPLACE TEMPORARY VIEW stock_desc_lookup AS
SELECT
 StockCode,
 MAX(Description) AS Master_Description
FROM online_retail
WHERE Description IS NOT NULL
 AND Description NOT IN ('NA', '', 'nan', '?') -- tapis sisa awal
GROUP BY StockCode

%sql

READY

-- Check if description ada unique ke
SELECT
 StockCode,
 COUNT(DISTINCT Description) AS Total_Unique_Descriptions,
 COLLECT_SET(Description) AS List_Of_Descriptions
FROM online_retail
GROUP BY StockCode
HAVING COUNT(DISTINCT Description) > 1

StockCode	Total_Unique_Descriptions	List_Of_Descriptions
21249	2	WrappedArray(WOOD ANIMALS HEIGHT CHART STICKERS, WOODLAND HEIGHT CHART STICKERS)
21535	2	WrappedArray(RETRO SPOT SMALL MILK JUG, RED RETROSPOT SMALL MILK JUG)
21711	2	WrappedArray(FOLDING UMBRELLA WHITE/RED POLKADOT, FOLDING UMBRELLA WHITE/RED SPOT)
23236	4	WrappedArray(DOILEY BISCUIT TIN, DOILEY STORAGE TIN, STORAGE TIN VINTAGE DOILEY, STORAGE TIN VINTAGE
22529	2	WrappedArray(MAGIC DRAWING SLATE GO TO THE FAIR, MAGIC SLATE GO TO THE FAIR)
82486	2	WrappedArray(WOOD S/3 CABINET ANT WHITE FINISH, 3 DRAWER ANTIQUE WHITE WOOD CABINET)
23462	2	WrappedArray(ROCOO WALL MIRROR, ROCOCO WALL MIRROR WHITE)
23535	3	WrappedArray(WALL ART BICYCLE SAFETY, WALL ART BICYCLE SAFTEY, BICYCLE SAFTEY WALL ART)
48184	2	WrappedArrav(DOORMAT ENGL ISH ROSE, DOOR MAT ENGL ISH ROSE)

%sql

READY

-- 1. Redo Bina Lookup Table becauase output above still show duplicate product name
CREATE OR REPLACE TEMPORARY VIEW stock_desc_lookup AS
SELECT
 StockCode,
 MAX(Description) AS Master_Description
FROM online_retail
WHERE Description IS NOT NULL
 AND Description NOT IN ('NA', '', 'nan', '?', '????damages????', '?display?', 'display')
GROUP BY StockCode

READY

```
%sql
-- 2. Map nama produk dan tapis gross sales
CREATE OR REPLACE TEMPORARY VIEW online_retail_clean AS
SELECT
    main.Invoice,
    main.StockCode,
    COALESCE(lookup.Master_Description, main.Description, 'NA') AS Description,
    main.Quantity,
    main.InvoiceDate,
    main.Price,
    main.Customer_ID,
    main.Country,
    main.Total_Sales,
    main.Log_Total_Sales
FROM online_retail main
LEFT JOIN stock_desc_lookup lookup
ON main.StockCode = lookup.StockCode
WHERE main.Total_Sales > 0
```

READY

```
%sql
-- Check if description ada unique ke
SELECT
    StockCode,
    COUNT(DISTINCT Description) AS Total_Unique_Descriptions,
    COLLECT_SET(Description) AS List_Of_Descriptions
FROM online_retail
GROUP BY StockCode
HAVING COUNT(DISTINCT Description) > 1
```

StockCode

▼ Total_Unique_Descriptions

READY

```
%sql
-- check rows with duplicates
SELECT
    Invoice,
    StockCode,
    Description,
    Quantity,
    InvoiceDate,
    Price,
    Customer_ID,
    Country,
    COUNT(*) as row_count
FROM online_retail_cleaned
GROUP BY
    Invoice,
    StockCode,
    Description,
    Quantity,
    InvoiceDate,
    Price,
    Customer_ID,
    Country
HAVING COUNT(*) > 1
```

READY

Invoice	StockCode	Description	Quantity	li
490300	21738	COSY SLIPPER SHOES SMALL RED	1	2
491204	47591B	SCOTTIES DES CHILD'S APRON	2	2
496651	47568	ENGLISH ROSE DESIGN PEG BAG	1	2
498567	21976	PACK OF 60 MUSHROOM CAKE CASES	1	2
499973	85123A	WHITE HANGING HEART T-LIGHT HOLDER	2	2
501900	21897	POTTING SHED CANDLE CITRONELLA	1	2
502189	21559	STRAWBERRY LUNCHBOX WITH CUTLERY	1	2
502660	17021	NAMASTE SWAGAT INCENSE	18	2
505177	22410	TIN CAN HOUSEKEEPING DESIGN	2	2

Output is truncated to 1000 rows. Learn more about `zeppelin.spark.maxResult`

```
%sql
-- sum the total number of duplicate rows
WITH duplicated_groups AS (
  SELECT COUNT(*) as row_count
  FROM online_retail_cleaned
  GROUP BY
    Invoice,
    StockCode,
    Description,
    Quantity,
    InvoiceDate,
    Price,
    Customer_ID,
    Country
  HAVING COUNT(*) > 1
)
SELECT SUM(row_count) AS total_duplicate_rows
FROM duplicated_groups
```

READY

total_duplicate_rows

66098

```
%sql
-- Drop duplicates & simpan ke dalam view baru yang bersih
CREATE OR REPLACE TEMPORARY VIEW online_retail_final AS
SELECT DISTINCT
  Invoice,
  StockCode,
  Description,
  Quantity,
  InvoiceDate,
  Price,
  Customer_ID,
  Country
FROM online_retail_cleaned
```

READY

```
%sql
-- recheck duplicate
WITH duplicated_groups AS (
  SELECT COUNT(*) as row_count
  FROM online_retail_final
  GROUP BY
    Invoice, StockCode, Description, Quantity, InvoiceDate, Price, Customer_ID, Country
  HAVING COUNT(*) > 1
)
SELECT COALESCE(SUM(row_count), 0) AS total_duplicate_rows
FROM duplicated_groups
```

READY

total_duplicate_rows

0

```
%pyspark
from pyspark.sql import functions as F

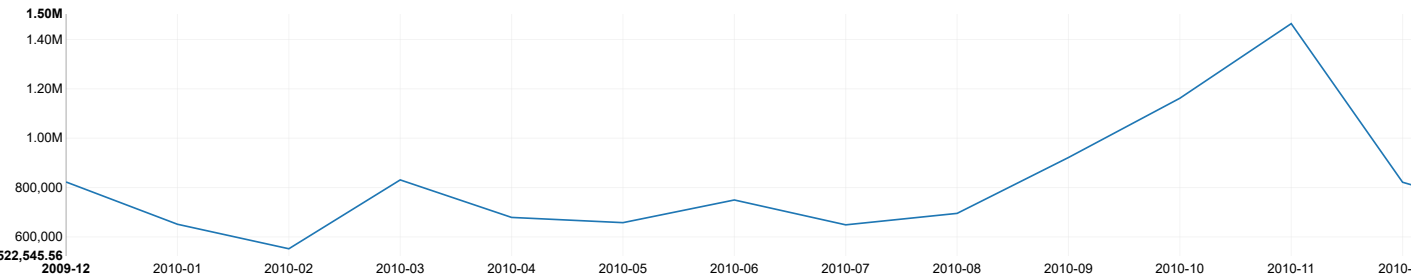
#Take clean data
df_clean = spark.table("online_retail_final")

##Feature Enginerring dalam zappelin gunenue pyspark
df_clean = df_clean.withColumn("Order_Month", F.date_format(F.col("InvoiceDate"), "yyyy-MM")) \
                    .withColumn("Order_Date", F.to_date(F.col("InvoiceDate"))) \
                    .withColumn("Order_Hour", F.hour(F.col("InvoiceDate"))) \
                    .withColumn("Day_Of_Week", F.dayofweek(F.col("InvoiceDate"))) \
                    .withColumn("Day_Type", F.when(F.col("Day_Of_Week").isin(1, 7), "Weekend").otherwise("Weekday")) \
                    .withColumn("Time_Period",
                                F.when((F.col("Order_Hour") >= 5) & (F.col("Order_Hour") < 12), "Morning")
                                  .when((F.col("Order_Hour") >= 12) & (F.col("Order_Hour") < 18), "Afternoon")
                                  .when((F.col("Order_Hour") >= 18) & (F.col("Order_Hour") < 23), "Evening")
                                  .otherwise("Late Night"))
                                )

##put as %sql table
df_clean.createOrReplaceTempView("online_retail_final_metrics")
```

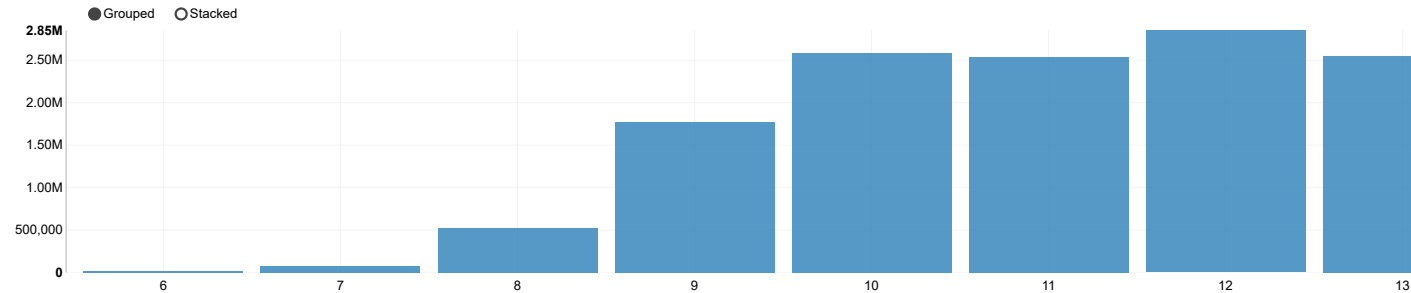
```
%sql
-- Compute Gross sales by Month
SELECT Order_Month, SUM(Price * Quantity) AS Total_Sale
FROM online_retail_final_metrics
GROUP BY Order_Month
ORDER BY Order_Month
```

settings

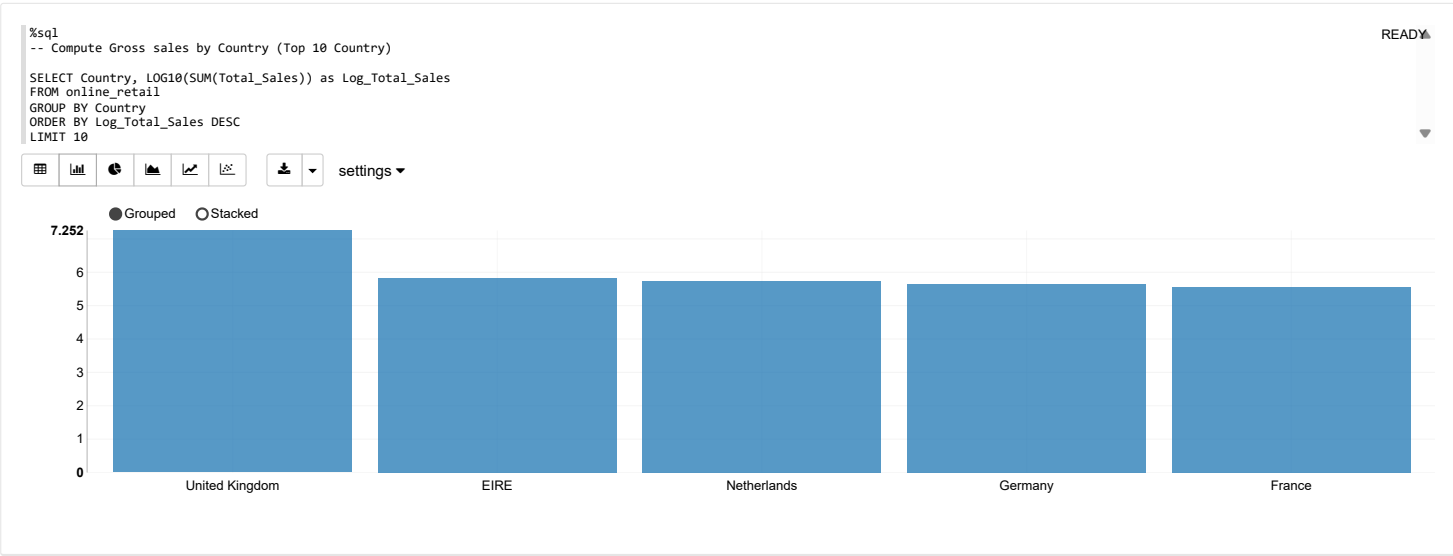
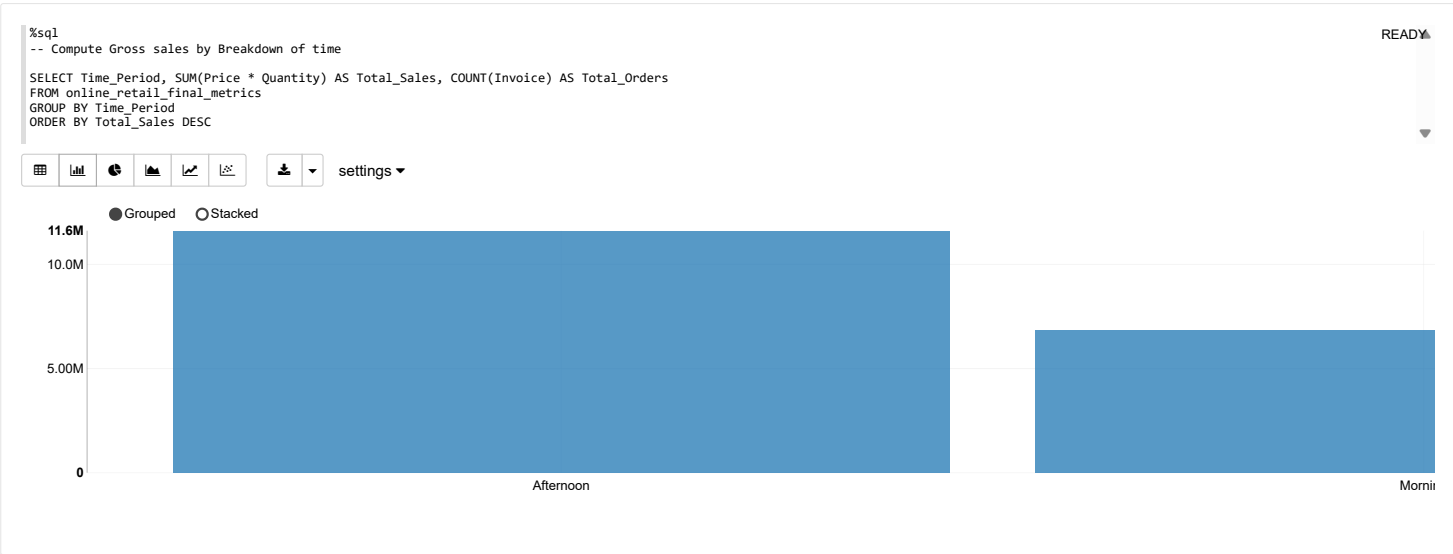
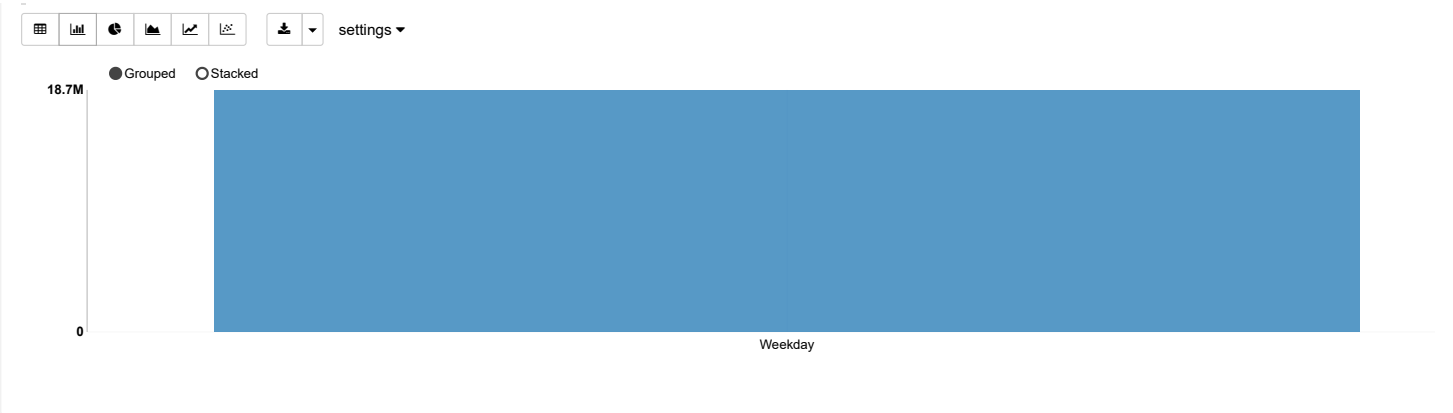


```
%sql
-- Compute Gross sales by Hour
SELECT Order_Hour, SUM(Price * Quantity) AS Total_Sale
FROM online_retail_final_metrics
GROUP BY Order_Hour
ORDER BY Order_Hour
```

settings



```
%sql
-- Compute Gross sales by Week of Day
SELECT Day_Type, SUM(Price * Quantity) AS Total_Sales, COUNT(Invoice) AS Total_Orders
FROM online_retail_final_metrics
```



Country	rank	StockCode	Product_Description
Australia	1	22492	MINI PAINT SET VINTAGE BOY+ GIRL
Australia	2	23084	RABBIT NIGHT LIGHT
Australia	3	21915	RED HARMONICA IN BOX
Australia	4	21731	RED TOADSTOOL LED NIGHT LIGHT
Australia	5	22969	HOMEMADE JAM SCENTED CANDLES
Australia	6	22630	DOLLY GIRL LUNCH BOX
Australia	7	22629	SPACEBOY LUNCH BOX
Australia	8	22544	MINI JIGSAW SPACEBOY
Australia	9	22539	MINI JIGSAW DOLLY GIRL

%sql
-- Compute Purchase Trend Similarities between country

WITH country_top_10 AS (
 SELECT
 Country,
 StockCode,
 MAX(Description) AS Product_Description,
 SUM(Quantity) AS Total_Quantity_Sold,
 ROW_NUMBER() OVER (PARTITION BY Country ORDER BY SUM(Quantity) DESC) as rank
 FROM online_retail_final_metrics
 WHERE Quantity > 0 AND Description != 'NA'
 GROUP BY Country, StockCode
)
SELECT
 StockCode,
 MAX(Product_Description) AS Product_Name,
 COUNT(DISTINCT Country) AS Number_Of_Countries_As_Top_10,
 COLLECT_SET(Country) AS List_Of_Countries
FROM country_top_10
WHERE rank <= 10
GROUP BY StockCode
HAVING COUNT(DISTINCT Country) > 1
ORDER BY Number_Of_Countries_As_Top_10 DESC

READY

StockCode	Product_Name	Number_Of_Countries_As_Top_10	List_Of_Countries
21240	SELECTOR RABBIT FOOT	2	WrappedArray(RSA, Hong Kong)
22423	REGENCY CAKESTAND 3 TIER	2	WrappedArray(Brazil, Lebanon)
21915	RED HARMONICA IN BOX	2	WrappedArray(Australia, Lithuania)
17003	BROCADE RING PURSE	2	WrappedArray(West Indies, United Kingdom)
22307	GOLD MUG BONE CHINA TREE OF LIFE	2	WrappedArray(Malta, Lithuania)
22582	PACK OF 6 SWEETIE GIFT BOXES	2	WrappedArray(Austria, Nigeria)
20677	PINK SPOTTY BOWL	2	WrappedArray(RSA, Hong Kong)
21928	JUMBO BAG SCANDINAVIAN PAISLEY	2	WrappedArray(Portugal, Channel Islands)
23077	DOUGHNUT LIP GLOSS	2	WrappedArray(Bahrain, Finland)

%sql
-- Compute top spender (High Value) customer
SELECT
 Customer_ID,
 Country,
 COUNT(DISTINCT Invoice) AS Total_Orders,
 SUM(Quantity) AS Total_Items_Bought,
 ROUND(SUM(Price * Quantity), 2) AS Total_Amount_Spent
FROM online_retail_final_metrics
WHERE Customer_ID IS NOT NULL
 AND Price > 0
 AND Quantity > 0
GROUP BY Customer_ID, Country
ORDER BY Total_Amount_Spent DESC
LIMIT 10

READY

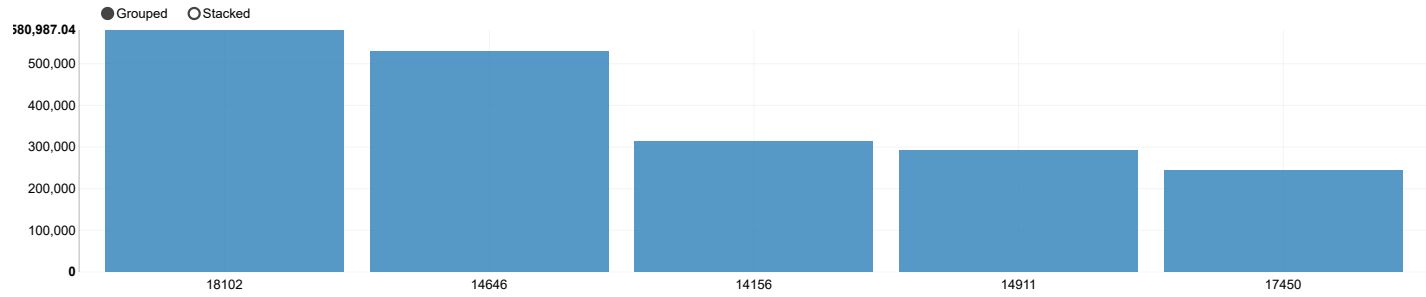
settings ▲

All fields:
Customer_ID Country Total_Orders Total_Items_Bought Total_Amount_Spent

Keys
Customer_ID ✕

Groups

Values
Total_Amount_Spent SUM ✕



%sql

-- Compute AOV by Country

WITH invoice_totals AS (
 SELECT
 Country,
 Invoice,
 SUM(Price * Quantity) AS Invoice_Total_Value
 FROM online_retail_final_metrics
 WHERE Price > 0 AND Quantity > 0
 GROUP BY Country, Invoice
)
SELECT
 Country,
 COUNT(Invoice) AS Total_Invoices,
 ROUND(AVG(Invoice_Total_Value), 2) AS Average_Order_Value,
 ROUND(SUM(Invoice_Total_Value), 2) AS Total_Country_Revenue
FROM invoice_totals
GROUP BY Country
HAVING Total_Invoices > 10
ORDER BY Average_Order_Value DESC

READY

Country	Total_Invoices	Average_Order_Value
Netherlands	228	2429.99
Singapore	11	2301.55
Australia	95	1781.93
Denmark	43	1594.9
Hong Kong	15	1579.03
Japan	33	1303.75
Norway	45	1251.61
Switzerland	93	1082.64
Greece	18	1060.9

%sql

-- Top Country is UK but Top AOV is Netherlands
-- Check if really UK is mostly B2C and Netherlands B2B

WITH invoice_summary AS (
 SELECT
 Country,
 Invoice,
 SUM(Quantity) AS Total_Items_In_Basket,
 SUM(Price * Quantity) AS Invoice_Value
 FROM online_retail_final_metrics
 WHERE Price > 0 AND Quantity > 0
 GROUP BY Country, Invoice
)
SELECT
 Country,
 COUNT(Invoice) AS Total_Invoices,
 ROUND(AVG(Total_Items_In_Basket), 0) AS Avg_Items_Per_Invoice,
 ROUND(AVG(Invoice_Value), 2) AS Average_Order_Value
FROM invoice_summary
WHERE Country IN ('United Kingdom', 'Netherlands')
GROUP BY Country

READY

Country	Total_Invoices	Avg_Items_Per_Invoice
United Kingdom	36536	251.0
Netherlands	228	1684.0

%sql

-- Top purchase item bulk purchase in Netherlands

SELECT

READY

StockCode,
MAX(Description) AS Product_Description,
SUM(Quantity) AS Total_Quantity_Sold,
COUNT(DISTINCT Invoice) AS How_Many_Invoices,
ROUND(SUM(Price * Quantity), 2) AS Total_Revenue

FROM online_retail_final_metrics
WHERE Country = 'Netherlands'
AND Quantity > 0
AND Description != 'NA'
GROUP BY StockCode
ORDER BY Total_Quantity_Sold DESC

settings ▲

All fields:

StockCode

Product_Description

Total_Quantity_Sold

How_Many_Invoices

Total_Revenue

Keys

Product_Description ✕

Groups

Values

Total_Quantity_Sold SUM ✕

● Grouped ○ Stacked

6,697

6,000

5,000

4,000

3,000

2,000

1,000

0

FOLKART ZINC HEART CHRISTMAS DEC

DOLLY GIRL LUNCH BOX

SPACEBOY LUNCH BOX

RED TOADSTOOL LED NIGHT LIGHT

PACK OF 72 RETROSPOT CAKE CASESROI

%sql

READY

localhost:9995/#!/notebook/2MX2N5T9T

10/10